

Unit-05

ASP.NET

What is ASP.NET?

ASP.NET is a web development framework developed by Microsoft for building dynamic web applications, websites, and services. It is part of the .NET framework and provides tools and libraries for creating robust and scalable web-based solutions.

5.1. ASP.NET vs ASP.NET Core

ASP.NET and ASP.NET Core are frameworks developed by Microsoft for building web applications. While both are used for similar purposes, ASP.NET Core is a newer, redesigned version of ASP.NET with improved performance, cross-platform capabilities, and modularity.

Differences Between ASP.NET and ASP.NET Core

Aspect	ASP.NET	ASP.NET Core
Platform Support	Windows-only.	Cross-platform (Windows, macOS, Linux).
Performance	Slower compared to ASP.NET Core.	High performance due to optimized runtime.
Dependency Injection	Limited support via third-party tools.	Built-in dependency injection.
Hosting	IIS (Internet Information Services) only.	Supports IIS, Kestrel, and self-hosting.
Configuration	XML-based configuration files.	JSON-based configuration (appsettings.json).
Development Workflow	Web Forms, MVC, and Web API as separate technologies.	Unified framework for MVC and Web API.
Deployment	Complex setup and deployment.	Simplified deployment with self-contained apps.
Open Source	Partially open source.	Fully open source and community-driven.

5.2. ASP.NET Frameworks for Web Applications

ASP.NET provides multiple frameworks to build web applications, each catering to different development styles and project needs. These frameworks include Web Forms, ASP.NET MVC, and ASP.NET Web Pages.

1. Web Forms

ASP.NET Web Forms is a framework designed for building form-based web applications with a drag-and-drop user interface. It simplifies development by abstracting HTML and JavaScript complexities.

Features of Web Forms:

- Event-driven programming model.
- Rapid development with Visual Studio's drag-and-drop controls.
- Built-in state management using ViewState.
- Ideal for developers transitioning from desktop to web development.

2. ASP.NET MVC

ASP.NET MVC (Model-View-Controller) is a framework for building scalable, testable, and maintainable web applications. It separates application logic, UI, and data to promote clean architecture.

Features of ASP.NET MVC:

- Clear separation of concerns (Model, View, Controller).
- Full control over HTML, CSS, and JavaScript.
- SEO-friendly URLs.
- Unit testing support.
- Extensible and customizable.

3. ASP.NET Web Pages

ASP.NET Web Pages is a lightweight framework for building dynamic, data-driven web applications using Razor syntax. It is ideal for small-scale applications.

Features of ASP.NET Web Pages:

- Razor syntax for embedding server code directly in HTML.
- Simple and fast development for small applications.
- Easy integration with databases like SQL Server.
- Minimal configuration required.

5.3. Creating a Simple Web Forms Application

Code for Default.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Default.aspx.cs"
Inherits="SimpleWebFormsApp.Default" %>

<!DOCTYPE html>

<html>

<head runat="server">

    <title>Simple Web Form</title>

</head>

<body>

    <form id="form1" runat="server">

        <div>

            <h2>Welcome to ASP.NET Web Forms</h2>

            <asp:Label ID="Label1" runat="server" Text="Enter Your Name:"></asp:Label>

            <asp:TextBox ID="TextBox1" runat="server"></asp:TextBox>

            <asp:Button ID="Button1" runat="server" Text="Submit" OnClick="Button1_Click" />

            <asp:Label ID="Label2" runat="server" ForeColor="Green"></asp:Label>

        </div>

    </form>

</body>

</html>
```

Default.aspx.cs

```
using System;

namespace SimpleWebFormsApp
{
    public partial class Default : System.Web.UI.Page
    {
        protected void Button1_Click(object sender, EventArgs e)
```

```
{  
    string name = TextBox1.Text;  
    Label2.Text = $"Hello, {name}! Welcome to ASP.NET Web Forms."  
}  
}  
}
```

5.4. Understanding ASP.NET MVC Architecture

ASP.NET MVC (Model-View-Controller) is a design pattern used to build scalable and maintainable web applications. It separates an application into three interconnected components: **Model**, **View**, and **Controller**, promoting organized code and reusability.

Components of ASP.NET MVC Architecture

1. Model

- Represents the data and business logic of the application.
- Responsible for retrieving, storing, and processing data from the database or other data sources.
- It typically contains classes and methods for data manipulation.

Example:

```
public class Product  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public decimal Price { get; set; }  
}
```

2. View

- Represents the user interface (UI) of the application.
- Displays data provided by the controller to the user.
- Typically contains HTML, CSS, and Razor syntax for dynamic content rendering.

Example:

```
@model List<Product>

<h1>Product List</h1>

<ul>

    @foreach (var product in Model)
    {
        <li>@product.Name - $@product.Price</li>
    }
</ul>
```

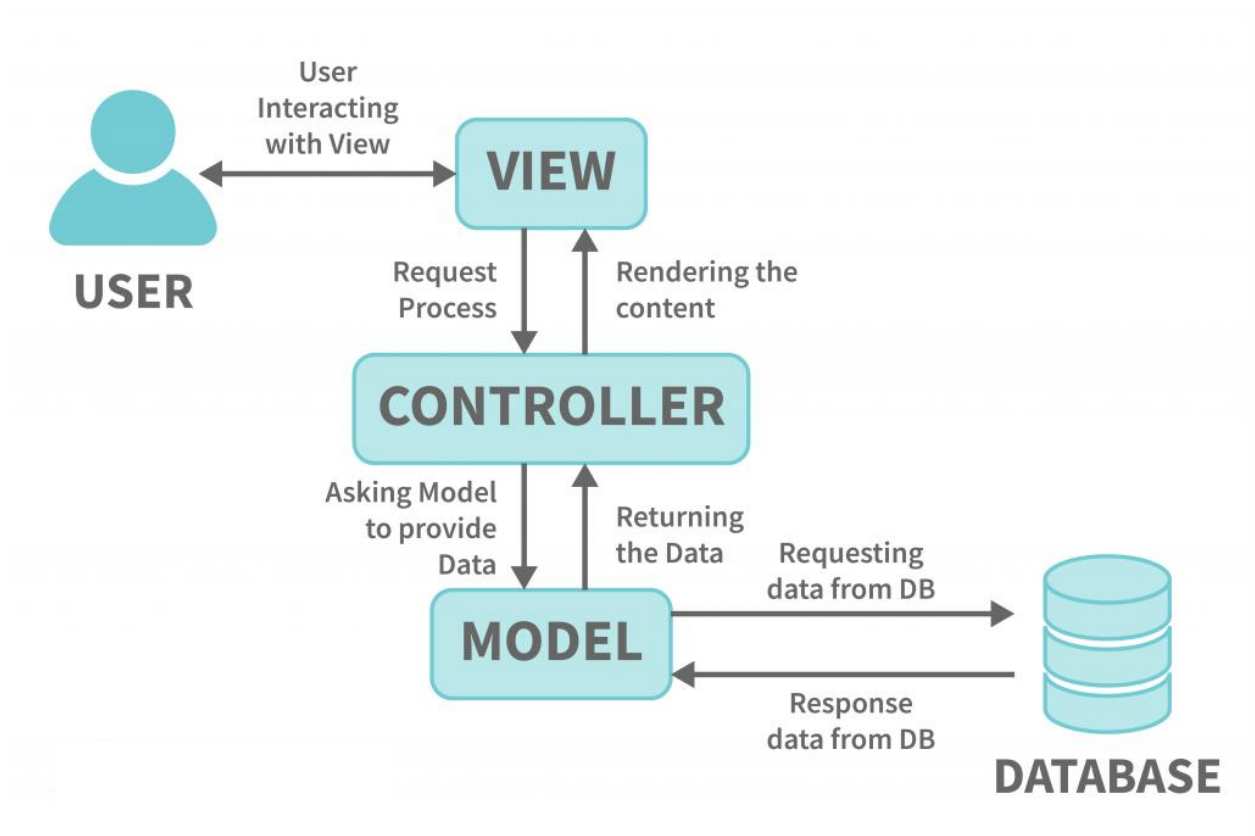
3. Controller

- Acts as the mediator between the Model and the View.
- Handles user input, processes requests, and decides which view to render.
- Uses methods (actions) to handle specific operations.

Example:

```
using Microsoft.AspNetCore.Mvc;

public class ProductController : Controller
{
    public IActionResult Index()
    {
        var products = new List<Product>
        {
            new Product { Id = 1, Name = "Laptop", Price = 1200 },
            new Product { Id = 2, Name = "Phone", Price = 800 }
        };
        return View(products);
    }
}
```



5.5 Creating Models, Views, Controllers, and URL Routing in ASP.NET MVC

1. Creating a Model

A **Model** in ASP.NET MVC represents the application's data and business logic.

- **Example:**

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
}
```

2. Creating a Controller

A **Controller** handles user requests, interacts with the Model, and selects a View to render.

- **Example:**

```
using System.Web.Mvc;

public class StudentController : Controller
```

```

{
    public ActionResult Index()
    {
        // Creating a list of students
        var students = new List<Student>
        {
            new Student { Id = 1, Name = "John", Age = 20 },
            new Student { Id = 2, Name = "Jane", Age = 22 }
        };
        // Passing the data to the view
        return View(students);
    }
}

```

3. Creating a View

A **View** displays data to the user.

- **Steps:**
 - Right-click on the Index action in the StudentController.
 - Select **Add View**.
 - Choose **Razor View** and name it Index.cshtml.
- **Example** (Index.cshtml):

```

@model List<Student>
<h2>Student List</h2>
<table>
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Age</th>
        </tr>

```

```

</thead>

<tbody>

    @foreach (var student in Model)
    {
        <tr>

            <td>@student.Id</td>

            <td>@student.Name</td>

            <td>@student.Age</td>

        </tr>

    }

</tbody>

</table>

```

4. URL Routing

ASP.NET MVC uses **Routing** to map URLs to the appropriate controllers and actions.

- **Default Route** (RouteConfig.cs):

```

routes.MapRoute(
    name: "Default",
    url: "{controller}/{action}/{id}",
    defaults: new { controller = "Home", action = "Index", id = UrlParameter.Optional }
);

```

5.6 Creating a Basic Layout for ASP.NET Web Pages

ASP.NET Web Pages provides a simple way to create dynamic web pages with Razor syntax. A layout in ASP.NET defines a consistent structure for your web application, such as headers, footers, and navigation.

Create a Layout Page

Example Layout Code (Layout.cshtml):

```

<!DOCTYPE html>

<html lang="en">

```



```

<head>

    <meta charset="utf-8" />

    <title>@ViewBag.Title</title>

    <link rel="stylesheet" href="~/Content/site.css" />

</head>

<body>

    <header>

        <h1>My ASP.NET Website</h1>

        <nav>

            <ul>

                <li><a href="/">Home</a></li>

                <li><a href="/About">About</a></li>

                <li><a href="/Contact">Contact</a></li>

            </ul>

        </nav>

    </header>

    <main>

        @RenderBody()

    </main>

    <footer>

        <p>&copy; @DateTime.Now.Year - My ASP.NET Website</p>

    </footer>

</body>

</html>

```

Apply the Layout to Pages

- In any Razor view file (e.g., Index.cshtml), specify the layout at the top.
- **Example** (Index.cshtml):

@page

```
@{
    Layout = "~/Views/Shared/_Layout.cshtml";
    ViewBag.Title = "Home Page";
}
<h2>Welcome to My Website</h2>
<p>This is the home page content.</p>
```

5.7 Razor Syntax for Embedding Code in HTML

Razor is a templating engine in ASP.NET that allows developers to embed server-side C# code within HTML. Razor is a lightweight, clean, and efficient markup syntax for embedding server-side code in HTML. It's primarily used in ASP.NET MVC and Razor Pages to create dynamic web pages. It is clean, lightweight, and makes it easy to work with data-driven web applications.

Variables in Razor

You can declare and use variables directly in Razor views.

Declaration:

```
@{
    var message = "Welcome to Razor Syntax!";
    int count = 5;
}
```

Usage:

```
<p>@message</p>
<p>Number of items: @count</p>
```

Loops in Razor

Razor supports loops like for, foreach, and while to iterate over data.

For Loop:

```
@for (int i = 1; i <= 5; i++)
{
    <p>Number: @i</p>
}
```

Foreach Loop:

```
@{
    var fruits = new List<string> { "Apple", "Banana", "Cherry" };
}
<ul>
    @foreach (var fruit in fruits)
    {
        <li>@fruit</li>
    }
</ul>
```

While Loop:

```
@{
    int count = 1;
}
@while (count <= 3)
{
    <p>Count: @count</p>
    count++;
}
```

Logical Expressions in Razor

Razor allows conditional statements such as if, else if, and else.

If Statement:

```
@if (DateTime.Now.Hour < 12)
{
    <p>Good Morning!</p>
}
```

Else If and Else:

```
@if (DateTime.Now.Hour < 12)
{
```

```
<p>Good Morning!</p>
}
else if (DateTime.Now.Hour < 18)
{
    <p>Good Afternoon!</p>
}
else
{
    <p>Good Evening!</p>
}
```

5.8 Database Interaction with ASP.NET MVC

ASP.NET MVC allows seamless interaction with databases using Entity Framework (EF) or other data access technologies. Below are the steps to implement database interaction in an ASP.NET MVC application.

Steps to Interact with the Database

Step 1: Create a Database

- Use SQL Server Management Studio (SSMS) or another database tool to create a database. Example:

```
CREATE DATABASE SchoolDB;
```

```
USE SchoolDB;
```

```
CREATE TABLE Students (
```

```
    Id INT PRIMARY KEY IDENTITY,
```

```
    Name NVARCHAR(50),
```

```
    Age INT,
```

```
    Email NVARCHAR(50)
```

```
);
```

Step 2: Add Entity Framework

1. Open the NuGet Package Manager in Visual Studio.
2. Install the Entity Framework package:

Install-Package EntityFramework

Step 3: Create a Model Class

The model represents the database table in the application.

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
    public string Email { get; set; }
}
```

Step 4: Create the DbContext Class

The DbContext class manages database operations and connections.

```
using System.Data.Entity;

public class SchoolContext : DbContext
{
    public DbSet<Student> Students { get; set; }
}
```

Step 5: Create the Controller

Use the StudentController to handle database interactions.

```
using System.Linq;
using System.Web.Mvc;

public class StudentController : Controller
{
}
```

```
private SchoolContext db = new SchoolContext();
```

```
// Display the list of students
```

```
public ActionResult Index()
```

```
{
```

```
    var students = db.Students.ToList();
```

```
    return View(students);
```

```
}
```

```
// Add a new student
```

```
[HttpGet]
```

```
public ActionResult Create()
```

```
{
```

```
    return View();
```

```
}
```

```
[HttpPost]
```

```
public ActionResult Create(Student student)
```

```
{
```

```
    if (ModelState.IsValid)
```

```
    {
```

```
        db.Students.Add(student);
```

```
        db.SaveChanges();
```

```
        return RedirectToAction("Index");
```

```
    }
```

```
    return View(student);
```

```
}
```

```
// Edit an existing student
```

```
[HttpGet]
```

```
public ActionResult Edit(int id)
```

```
{
```

```

var student = db.Students.Find(id);

if (student == null)
{
    return HttpNotFound();
}

return View(student);
}

[HttpPost]
public ActionResult Edit(Student student)
{
    if (ModelState.IsValid)
    {
        db.Entry(student).State = System.Data.Entity.EntityState.Modified;
        db.SaveChanges();
        return RedirectToAction("Index");
    }

    return View(student);
}

// Delete a student

[HttpGet]
public ActionResult Delete(int id)
{
    var student = db.Students.Find(id);
    if (student == null)
    {
        return HttpNotFound();
    }

    return View(student);
}

```

```
[HttpPost, ActionName("Delete")]
public ActionResult DeleteConfirmed(int id)
{
    var student = db.Students.Find(id);
    db.Students.Remove(student);
    db.SaveChanges();
    return RedirectToAction("Index");
}
}
```

Step 6: Create Views

Create corresponding views for actions like Index, Create, Edit, and Delete.

- **Index.cshtml**

```
@model IEnumerable<Student>

<h2>Students</h2>

<table>

    <thead>

        <tr>

            <th>Name</th>

            <th>Age</th>

            <th>Email</th>

            <th>Actions</th>

        </tr>

    </thead>

    <tbody>

        @foreach (var student in Model)
        {
            <tr>
```



```
<td>@student.Name</td>

<td>@student.Age</td>

<td>@student.Email</td>

<td>

    @Html.ActionLink("Edit", "Edit", new { id = student.Id }) |

    @Html.ActionLink("Delete", "Delete", new { id = student.Id })

</td>

</tr>

}

</tbody>

</table>

<a href="@Url.Action("Create", "Student")">Add New Student</a>
```